```python
"""
app.py — Dashboard Benchmark Bancario Chile
Ejecutar: streamlit run app.py
"""
import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
import streamlit as st
import kpis as K
import nim_predictor as NP

# ──────────────────────────────────────────────────────────────────────────
# CONFIGURACIÓN
# ──────────────────────────────────────────────────────────────────────────
st.set_page_config(page_title="Benchmark Bancario Chile", page_icon="🏦",
                   layout="wide", initial_sidebar_state="expanded")

st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Open+Sans:wght@400;600;700&display=swap'
html, body, [class*="css"] { font-family: 'Open Sans', sans-serif; background:#f4f4f4; }

[data-testid="stSidebar"] { background:#1a1a2e; }
[data-testid="stSidebar"] * { color:#e8edf2 !important; }
[data-testid="stSidebar"] .stSelectbox label {
    color:#F5A623 !important; font-size:0.72rem;
    text-transform:uppercase; letter-spacing:0.08em; font-weight:700;
}
.main .block-container { padding-top:0.8rem; padding-bottom:2rem; max-width:1440px; }

.dash-header {
    background:linear-gradient(135deg,#1a1a2e 0%,#16213e 100%);
    border-radius:8px; padding:1.1rem 1.8rem; margin-bottom:1rem;
    border-left:6px solid #F5A623;
}
.dash-header h1 { color:#fff; font-size:1.4rem; font-weight:700; margin:0; }
.dash-header p  { color:#94a9be; font-size:0.8rem; margin:0.2rem 0 0 0; }

.section-title {
    font-size:0.82rem; font-weight:700; color:#F5A623;
    text-transform:uppercase; letter-spacing:0.05em;
    padding:0.35rem 0; margin-bottom:0.7rem;
    border-bottom:2px solid #F5A623;
}

/* Tabla de resultados */
.res-table { border-collapse:collapse; width:100%; font-size:0.8rem; }
.res-table th { padding:0.35rem 0.5rem; text-align:center; font-weight:700;
                color:#fff; font-size:0.72rem; white-space:nowrap; }
.res-table td { padding:0.28rem 0.5rem; text-align:right;
                border-bottom:1px solid #e8edf2; }
```

```
.res-table td:first-child { text-align:left; font-weight:600; white-space:nowrap; }
.res-table tr.total-row td { font-weight:700; background:#f0f0f0;
                              border-top:2px solid #ccc; }
.res-table tr.total-row td:first-child { color:#1a1a2e; }
</style>
""", unsafe_allow_html=True)

# ────────────────────────────────────────────────────────────────────────────
# FORMATO NUMÉRICO
# ────────────────────────────────────────────────────────────────────────────
def cl(val, dec=0):
    """Número con formato chileno (puntos miles, coma decimal)."""
    if val is None or (isinstance(val, float) and np.isnan(val)):
        return "—"
    s = f"{val:,.{dec}f}".replace(",","X").replace(".",",").replace("X",".")
    return s

def mmm(val):
    """MMM$ sin decimales."""
    if val is None or (isinstance(val, float) and np.isnan(val)): return "—"
    return f"MMM$ {cl(val, 0)}"

def pct(val, dec=2):
    if val is None or (isinstance(val, float) and np.isnan(val)): return "—"
    return f"{cl(val, dec)}%"

def delta_html(val, es_pct=False):
    if val is None or (isinstance(val, float) and np.isnan(val)):
        return '<span style="color:#888">—</span>'
    color = "#2E7D32" if val > 0 else ("#CC0000" if val < 0 else "#888")
    arrow = "▲" if val > 0 else ("▼" if val < 0 else "●")
    txt = f"{cl(abs(val),1)}%" if es_pct else cl(val, 0)
    return f'<span style="color:{color};font-weight:700">{arrow} {txt}</span>'

# ────────────────────────────────────────────────────────────────────────────
# GRÁFICOS — helpers
# ────────────────────────────────────────────────────────────────────────────
COLORES = K.COLORES_BANCO
FILL    = {
    "BancoEstado":    "rgba(245,166,35,0.10)",
    "Banco de Chile": "rgba(26,95,168,0.10)",
    "Santander":      "rgba(204,0,0,0.10)",
    "BCI":            "rgba(46,125,50,0.10)",
    "Sistema":        "rgba(136,136,136,0.10)",
}
PLOT_CFG = dict(displayModeBar=False, responsive=True)

MESES_ES = {
    "Jan":"Ene","Feb":"Feb","Mar":"Mar","Apr":"Abr","May":"May","Jun":"Jun",
    "Jul":"Jul","Aug":"Ago","Sep":"Sep","Oct":"Oct","Nov":"Nov","Dec":"Dic"
}

def tickvals_es(periodos):
    """Genera tickvals y ticktext con meses en español para un índice de fechas."""
    if len(periodos) == 0:
        return [], []
```

```
    tickvals = list(periodos)
    ticktext = [f"{MESES_ES.get(p.strftime('%b'), p.strftime('%b'))} {p.strftime('%y')}" for p
    return tickvals, ticktext

def layout_base(title="", h=340, yaxis_visible=False, yaxis_suffix="", periodos=None, r=60):
    tv, tt = (tickvals_es(periodos) if periodos is not None else (None, None))
    xaxis = dict(showgrid=False, zeroline=False, linecolor="#ddd",
                 tickfont=dict(size=11, color="#333"))
    if tv:
        xaxis["tickvals"] = tv
        xaxis["ticktext"] = tt
    else:
        xaxis["tickformat"] = "%b %Y"

    return dict(
        paper_bgcolor="white", plot_bgcolor="white",
        font=dict(family="Open Sans", size=11, color="#1a1a2e"),
        height=h,
        margin=dict(l=10, r=r, t=45, b=40),
        title=dict(text=f"<b>{title}</b>", font=dict(size=13,color="#1a1a2e"), x=0),
        legend=dict(
            orientation="h", yanchor="bottom", y=1.02,
            xanchor="right", x=1,
            font=dict(size=12, color="#1a1a2e"),
            bgcolor="rgba(0,0,0,0)",
            borderwidth=0,
        ),
        xaxis=xaxis,
        yaxis=dict(visible=yaxis_visible, showgrid=False, zeroline=False,
                   ticksuffix=yaxis_suffix if yaxis_visible else ""),
    )

# Orden fijo de bancos
ORDEN_BANCOS_DISPLAY = K.ORDEN_BANCOS  # ["BancoEstado","Banco de Chile","Santander","BCI","Si

# Colores para tipos de colocación (distintos de colores banco)
COLORES_TIPO_COL = {
    "Comercial": "#2196F3",   # azul material
    "Vivienda":  "#9C27B0",   # morado
    "Consumo":   "#FF5722",   # naranja-rojo
}

POSICION_ETIQUETA = {
    "BancoEstado":    "top center",
    "Banco de Chile": "bottom center",
    "Santander":      "top center",
    "BCI":            "bottom center",
    "Sistema":        "top center",
}

def agregar_linea(fig, serie, banco, dash="solid", width=2.5, size=5, mostrar_etiquetas=True,
    """Agrega traza con etiquetas: primer punto izquierda, último derecha."""
    color = COLORES.get(banco, "#aaa")
    texts = [""] * len(serie)
    tpos  = ["middle left"] * len(serie)
    if mostrar_etiquetas and len(serie) > 0:
```

```
        suffix = "%" if fmt_dec == 2 else ""
        texts[0]  = cl(serie.iloc[0], fmt_dec) + suffix
        texts[-1] = cl(serie.iloc[-1], fmt_dec) + suffix
        tpos[0]   = "middle left"
        tpos[-1]  = "middle right"
    fig.add_trace(go.Scatter(
        x=serie.index, y=serie.values, name=banco,
        mode="lines+markers+text",
        line=dict(color=color, width=width, dash=dash),
        marker=dict(size=size, color=color),
        text=texts, textposition=tpos,
        textfont=dict(size=12, color=color),
        hovertemplate=f"<b>{banco}</b><br>%{{x|%b %Y}}: %{{y:,.{fmt_dec}f}}<extra></extra>",
    ))


def agregar_lineas_sin_superposicion(fig, series_dict, umbral_pct=3.0, size=5, fmt_dec=0):
    """Etiquetas: primer punto izquierda, último derecha, con desplazamiento vertical si hay s
    if not series_dict:
        return
    todos_vals = [v for s in series_dict.values() for v in s.values if pd.notna(v)]
    if not todos_vals:
        return
    rango  = max(todos_vals) - min(todos_vals)
    umbral = rango * umbral_pct / 100 if rango > 0 else 1
    primeros = {b: s.iloc[0]  for b, s in series_dict.items() if len(s) > 0}
    ultimos  = {b: s.iloc[-1] for b, s in series_dict.items() if len(s) > 0}

    def asignar_pos(vals_dict, lado):
        bancos_ord = sorted(vals_dict.keys(), key=lambda b: vals_dict[b], reverse=True)
        pos = {}
        for i, banco in enumerate(bancos_ord):
            if i == 0:
                pos[banco] = f"top {lado}"
            else:
                prev = bancos_ord[i - 1]
                if abs(vals_dict[banco] - vals_dict[prev]) < umbral:
                    pos[banco] = f"bottom {lado}" if pos[prev] == f"top {lado}" else f"top {la
                else:
                    pos[banco] = f"middle {lado}"
        return pos

    pos_inicio = asignar_pos(primeros, "left")
    pos_fin    = asignar_pos(ultimos,  "right")

    for banco, serie in series_dict.items():
        color = COLORES.get(banco, "#aaa")
        texts = [""] * len(serie)
        tpos  = ["middle left"] * len(serie)
        if len(serie) > 0:
            suffix = "%" if fmt_dec == 2 else ""
            texts[0]  = cl(serie.iloc[0], fmt_dec) + suffix
            texts[-1] = cl(serie.iloc[-1], fmt_dec) + suffix
            tpos[0]   = pos_inicio.get(banco, "middle left")
            tpos[-1]  = pos_fin.get(banco, "middle right")
        fig.add_trace(go.Scatter(
```

```
            x=serie.index, y=serie.values, name=banco,
            mode="lines+markers+text",
            line=dict(color=color, width=2.5),
            marker=dict(size=size, color=color),
            text=texts, textposition=tpos,
            textfont=dict(size=12, color=color),
            hovertemplate=f"<b>{banco}</b><br>%{{x|%b %Y}}: %{{y:,.{fmt_dec}f}}<extra></extra>
        ))

# ──────────────────────────────────────────────────────────────────────────────
def _bancos_ordenados(cols):
    orden = ["BancoEstado", "Banco de Chile", "Santander", "BCI", "Sistema"]
    return [b for b in orden if b in cols] + [b for b in cols if b not in orden]
def _leyenda_bancos():
    """Genera HTML de leyenda compartida para los 4 bancos."""
    items = "".join(
        f'<span style="display:inline-flex;align-items:center;margin-right:1.2rem">'
        f'<span style="display:inline-block;width:28px;height:3px;background:{COLORES.get(b,"#
        f'<span style="font-size:0.8rem;color:#333;font-family:Open Sans;font-weight:600">{b}<
        for b in K.ORDEN_BANCOS if b != K.NOMBRE_SISTEMA
    )
    return f'<div style="background:#fff;padding:0.5rem 0.8rem;border-radius:4px;margin-bottom


def _grafico_composicion(df_comp, colores_tipo, orden_tipos):
    """Donut (total 4 bancos) + barras apiladas (por banco). Reutilizable en cualquier tab."""
    if df_comp.empty:
        st.info("Sin datos para el período seleccionado.")
        return
    col_don, col_bar = st.columns([1, 2])

    with col_don:
        tot = (df_comp.groupby("tipo")["valor"].sum()
                      .reindex(orden_tipos).dropna())
        fig_don = go.Figure(go.Pie(
            labels=tot.index, values=tot.values, hole=0.52,
            marker_colors=[colores_tipo.get(t, "#aaa") for t in tot.index],
            textinfo="label+percent",
            textfont=dict(size=11, color="#1a1a2e"),
            insidetextorientation="radial",
            hovertemplate="%{label}: MMM$ %{value:,.0f}<extra></extra>",
        ))
        fig_don.update_layout(
            title=dict(text="4 Bancos — Total", font=dict(size=12, color="#1a1a2e"), x=0.5),
            height=320, margin=dict(l=10, r=10, t=45, b=10),
            showlegend=False,
            paper_bgcolor="white", plot_bgcolor="white",
            font=dict(family="Open Sans", size=11, color="#1a1a2e"),
        )
        st.plotly_chart(fig_don, use_container_width=True, config=PLOT_CFG)

    with col_bar:
        fig_stack = go.Figure()
        bancos_ord = [b for b in K.ORDEN_BANCOS if b in df_comp["banco"].unique()]
        for tipo in orden_tipos:
            vals = []
```

```
            for banco in bancos_ord:
                sub = df_comp[(df_comp["tipo"] == tipo) & (df_comp["banco"] == banco)]
                vals.append(sub["valor"].sum() if not sub.empty else 0)
            fig_stack.add_trace(go.Bar(
                name=tipo, x=bancos_ord, y=vals,
                marker_color=colores_tipo.get(tipo, "#aaa"),
                texttemplate="%{y:,.0f}", textposition="inside",
                textfont=dict(size=10, color="#fff"),
                hovertemplate=f"{tipo}: MMM$ %{{y:,.0f}}<extra></extra>",
            ))
        fig_stack.update_layout(
            barmode="stack",
            title=dict(text="Por banco (MMM$)", font=dict(size=12, color="#1a1a2e"), x=0.5),
            height=320, margin=dict(l=20, r=20, t=45, b=80),
            showlegend=True,
            legend=dict(orientation="h", y=-0.45, font=dict(size=10, color="#1a1a2e"),
                        bgcolor="white", borderwidth=0),
            paper_bgcolor="white", plot_bgcolor="white",
            font=dict(family="Open Sans", size=11, color="#1a1a2e"),
            xaxis=dict(showgrid=False, tickfont=dict(size=12, color="#1a1a2e")),
            yaxis=dict(visible=False, showgrid=False),
        )
        st.plotly_chart(fig_stack, use_container_width=True, config=PLOT_CFG)


# SIDEBAR Y CARGA
# ──────────────────────────────────────────────────────────────────────────────
@st.cache_data(show_spinner="Cargando datos CMF…")
def cargar(data_dir_str, moneda): return K.cargar_datos(Path(data_dir_str), moneda)

with st.sidebar:
    st.markdown("### 🏦 Benchmark Bancario")
    st.markdown("---")
    data_dir = st.text_input("Ruta de datos", value="data")

    st.markdown("**Moneda**")
    moneda_sel = st.radio("", ["Total","CLP","UF","MX"],
                          horizontal=True, label_visibility="collapsed")
    moneda_key = moneda_sel.lower()

    st.markdown("---")
    try:
        df_b1, df_r1 = cargar(data_dir, moneda_key)
        periodos = sorted(df_b1["periodo"].unique())
        DATA_OK = True
    except Exception as e:
        st.error(f"⚠️ {e}")
        DATA_OK = False
        periodos = []

    if DATA_OK:
        periodo_str = st.selectbox("Período",
            [p.strftime("%Y-%m") for p in periodos[::-1]], index=0)
        periodo_sel = pd.Timestamp(periodo_str)
        anio_desde  = st.selectbox("Desde año", [2025, 2024, 2023, 2022], index=0)
        fecha_desde = pd.Timestamp(f"{anio_desde}-01-01")
```

```
        df_b1_f = df_b1[df_b1["periodo"] >= fecha_desde]
        df_r1_f = df_r1[df_r1["periodo"] >= fecha_desde]

        st.markdown("---")
        st.markdown(
            '<span style="color:#F5A623;font-size:0.72rem;font-weight:700;'
            'text-transform:uppercase;letter-spacing:0.08em">Variación comparativa</span>',
            unsafe_allow_html=True)
        delta_modo = st.radio(
            "", ["Δ Mes", "Δ 12m", "Δ Año"],
            horizontal=False, label_visibility="collapsed",
            help="Δ Mes = vs mes anterior · Δ 12m = mismo mes año anterior · Δ Año = vs diciem

        st.markdown("---")
        st.markdown(f"<small style='color:#94a9be'>CMF Chile · {len(periodos)} períodos<br>"
                    f"Último: {periodos[-1].strftime('%b %Y')}<br>"
                    f"Moneda: {moneda_sel}</small>", unsafe_allow_html=True)

# ────────────────────────────────────────────────────────────────────────────
# HEADER
# ────────────────────────────────────────────────────────────────────────────
mes_lbl = pd.Timestamp(periodo_str).strftime("%B %Y").capitalize() if DATA_OK else "—"
st.markdown(f"""
<div class="dash-header">
  <h1>🏦 Análisis Industria Bancaria — Chile</h1>
  <p>4 Grandes Bancos · Datos CMF · {mes_lbl} · Cifras en MMM$ · Moneda: {moneda_sel if DATA_O
</div>""", unsafe_allow_html=True)

if not DATA_OK:
    st.warning("Configura la ruta de datos en el panel lateral.")
    st.stop()

# ── Período de referencia según filtro global de delta ──────────────────────
# Δ Mes  → mes anterior
# Δ 12m  → mismo mes año anterior
# Δ Año  → diciembre del año anterior (YTD)
def _periodo_ref(periodo: pd.Timestamp, modo: str) -> pd.Timestamp:
    if modo == "Δ Mes":
        return periodo - pd.DateOffset(months=1)
    elif modo == "Δ 12m":
        return periodo - pd.DateOffset(months=12)
    else:  # Δ Año
        return pd.Timestamp(f"{periodo.year - 1}-12-01")

periodo_ref  = _periodo_ref(periodo_sel, delta_modo)
delta_lbl    = delta_modo  # etiqueta para mostrar en columnas

tab1,tab2,tab_pas,tab3,tab3b,tab4,tab5,tab7 = st.tabs([
    "📊 Resumen","📈 Activos","📉 Pasivos","⚖️ Balance",
    "📋 Resultados","💰 Ingresos","📊 Rentabilidad","🔮 Predicción NIM"])

# ────────────────────────────────────────────────────────────────────────────
# HELPER: renderizar tabla de balance estilo presentación
# ────────────────────────────────────────────────────────────────────────────
def _render_tabla_balance(df_tab: pd.DataFrame, delta_lbl: str = "MTD"):
    """Renderiza tabla de balance con colores por banco y variaciones MTD."""
```

```python
bancos = list(K.BANCOS.values()) + [K.NOMBRE_SISTEMA]
color_hdrs = {
    "BancoEstado":    "#F5A623",
    "Banco de Chile": "#1A5FA8",
    "Santander":      "#CC0000",
    "BCI":            "#2E7D32",
    "Sistema":        "#444444",
}

# Encabezados
hdr_saldos = "".join(
    f'<th colspan="2" style="background:{color_hdrs.get(b,"#555")};'
    f'padding:0.3rem 0.4rem;color:#fff;font-size:0.72rem;text-align:center">{b}</th>'
    for b in bancos
)
sub_hdr = "".join(
    f'<th style="background:#e8e8e8;color:#555;font-size:0.68rem;'
    f'padding:0.2rem 0.4rem;text-align:right">Saldo</th>'
    f'<th style="background:#e8e8e8;color:#555;font-size:0.68rem;'
    f'padding:0.2rem 0.4rem;text-align:right">{delta_lbl}</th>'
    for _ in bancos
)

filas_html = ""
for _, row in df_tab.iterrows():
    es_total = row.get("es_total", False)
    nivel    = row.get("nivel", 1)
    linea    = row["Línea"]

    # Estilo según nivel
    if nivel == 0:
        bg    = "#1a1a2e"
        color = "#fff"
        fw    = "700"
        fs    = "0.85rem"
    elif es_total:
        bg    = "#3a3a6a"
        color = "#fff"
        fw    = "700"
        fs    = "0.83rem"
    else:
        bg    = "#fff"
        color = "#333"
        fw    = "400"
        fs    = "0.85rem"

    fila_html = (
        f'<tr style="background:{bg}">'
        f'<td style="padding:0.25rem 0.5rem;color:{color};font-weight:{fw};'
        f'font-size:{fs};white-space:nowrap;border-bottom:1px solid #eee">'
        f'{linea}</td>'
    )

    for banco in bancos:
        saldo = row.get(f"{banco}_saldo", np.nan)
        mtd   = row.get(f"{banco}_mtd",   np.nan)
```

```
            s_str = cl(saldo, 0) if pd.notna(saldo) else "—"
            m_str = cl(mtd,   0) if pd.notna(mtd)   else "—"
            m_color = "#00C853" if (pd.notna(mtd) and mtd > 0) else (
                      "#CC0000" if (pd.notna(mtd) and mtd < 0) else "#888")
            # Sistema: gris oscuro en totales, claro en detalle
            if banco == K.NOMBRE_SISTEMA:
                if nivel == 0:
                    bg_cel = "background:#3a3a3a;"
                    txt_cel = "#fff"
                elif es_total:
                    bg_cel = "background:#4a4a4a;"
                    txt_cel = "#fff"
                else:
                    bg_cel = "background:#f0f4ff;"
                    txt_cel = "#1a1a2e"
            else:
                bg_cel = ""
                txt_cel = color

            fila_html += (
                f'<td style="padding:0.25rem 0.5rem;text-align:right;'
                f'color:{txt_cel};font-weight:{fw};font-size:{fs};'
                f'border-bottom:1px solid #eee;{bg_cel}">{s_str}</td>'
                f'<td style="padding:0.25rem 0.5rem;text-align:right;'
                f'color:{m_color};font-weight:700;font-size:{fs};'
                f'border-bottom:1px solid #eee;{bg_cel}">{m_str}</td>'
            )

        fila_html += "</tr>"
        filas_html += fila_html

    tabla_html = f"""
    <div style="overflow-x:auto;margin-bottom:0.5rem">
    <table style="border-collapse:collapse;width:100%;font-size:0.8rem">
      <thead>
        <tr>
          <th style="background:#1a1a2e;color:#fff;padding:0.3rem 0.5rem;
                     text-align:left;font-size:0.75rem;min-width:180px"></th>
          {hdr_saldos}
        </tr>
        <tr>
          <th style="background:#1a1a2e;padding:0.2rem 0.5rem"></th>
          {sub_hdr}
        </tr>
      </thead>
      <tbody>{filas_html}</tbody>
    </table>
    </div>
    <small style="color:#888">Cifras en MMM$ · {delta_lbl} = variación respecto al período de
    """
    st.markdown(tabla_html, unsafe_allow_html=True)


def _render_tabla_resultados(df_tab: pd.DataFrame, delta_lbl: str = "Δ"):
    """Renderiza tabla de resultados con 3 niveles de jerarquía."""
    bancos = list(K.BANCOS.values()) + [K.NOMBRE_SISTEMA]
```

```
color_hdrs = {
    "BancoEstado":    "#F5A623",
    "Banco de Chile": "#1A5FA8",
    "Santander":      "#CC0000",
    "BCI":            "#2E7D32",
    "Sistema":        "#444444",
}
hdr = "".join(
    f'<th colspan="2" style="background:{color_hdrs.get(b,"#555")};'
    f'padding:0.3rem 0.5rem;color:#fff;font-size:0.72rem;text-align:center">{b}</th>'
    for b in bancos)
sub = "".join(
    f'<th style="background:#e8e8e8;color:#555;font-size:0.68rem;padding:0.2rem 0.4rem;tex
    f'<th style="background:#e8e8e8;color:#555;font-size:0.68rem;padding:0.2rem 0.4rem;tex
    for _ in bancos)

filas_html = ""
for _, row in df_tab.iterrows():
    nivel    = row.get("nivel", 2)
    es_total = row.get("es_total", False)
    linea    = row["Línea"]

    if nivel == 0:
        bg = "#1a1a2e"; fw = "700"; fs = "0.85rem"; color = "#fff"
    elif nivel == 1 or es_total:
        bg = "#3a3a6a"; fw = "700"; fs = "0.83rem"; color = "#fff"
    else:
        bg = "#fff";    fw = "400"; fs = "0.85rem"; color = "#333"

    filas_html += (
        f'<tr style="background:{bg}">'
        f'<td style="padding:0.25rem 0.6rem;color:{color};font-weight:{fw};'
        f'font-size:{fs};white-space:nowrap;border-bottom:1px solid #eee">{linea}</td>'
    )
    for banco in bancos:
        val  = row.get(f"{banco}_val",  np.nan)
        dval = row.get(f"{banco}_d12m", np.nan)
        vs   = cl(val,  0) if pd.notna(val)  and val  != 0 else "—"
        ds   = cl(dval, 0) if pd.notna(dval) and dval != 0 else "—"
        dc   = "#00C853" if (pd.notna(dval) and dval > 0) else (
               "#CC0000" if (pd.notna(dval) and dval < 0) else "#aaa")
        if banco == K.NOMBRE_SISTEMA:
            if nivel == 0:
                bg_cel = "background:#3a3a3a;"; txt_cel = "#fff"
            elif nivel == 1 or es_total:
                bg_cel = "background:#4a4a4a;"; txt_cel = "#fff"
            else:
                bg_cel = "background:#f0f4ff;"; txt_cel = "#1a1a2e"
        else:
            bg_cel = ""; txt_cel = color
        filas_html += (
            f'<td style="padding:0.25rem 0.5rem;text-align:right;color:{txt_cel};'
            f'font-weight:{fw};font-size:{fs};border-bottom:1px solid #eee;{bg_cel}">{vs}<
            f'<td style="padding:0.25rem 0.5rem;text-align:right;color:{dc};'
            f'font-weight:700;font-size:{fs};border-bottom:1px solid #eee;{bg_cel}">{ds}</
        )
```

```
        filas_html += "</tr>"

    _NOTAS = {
        "Δ Mes": "variación vs mes anterior",
        "Δ 12m": "variación vs mismo mes año anterior",
        "Δ Año": "variación vs diciembre año anterior",
    }
    st.markdown(f"""
    <div style="overflow-x:auto;margin-bottom:0.5rem">
    <table style="border-collapse:collapse;width:100%;font-size:0.8rem">
      <thead>
        <tr>
          <th style="background:#1a1a2e;color:#fff;padding:0.3rem 0.6rem;
                     text-align:left;font-size:0.75rem;min-width:200px"></th>
          {hdr}
        </tr>
        <tr>
          <th style="background:#1a1a2e;padding:0.2rem 0.5rem"></th>
          {sub}
        </tr>
      </thead>
      <tbody>{filas_html}</tbody>
    </table></div>
    <small style="color:#888">Cifras en MMM$ · {delta_lbl} = {_NOTAS.get(delta_lbl, "variación
    """, unsafe_allow_html=True)


# ═══════════════════════════════════════════════════════════════════════
# TAB 1 — RESUMEN
# ═══════════════════════════════════════════════════════════════════════
with tab1:
    col_t = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_COLOCACIONES))
    act_t = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_TOTAL_ACTIVOS))
    nim_t = K.calcular_nim(df_b1_f, df_r1_f)
    roe_t = K.calcular_roe(df_b1_f, df_r1_f)

    def _g(tabla, banco):
        try: return tabla.loc[periodo_sel, banco] if (periodo_sel in tabla.index and banco in t
        except: return np.nan

    def _mom(tabla, banco):
        try:
            idx = tabla.index.get_loc(periodo_sel)
            return tabla.loc[periodo_sel, banco] - tabla.iloc[idx-1][banco] if idx>0 else np.n
        except: return np.nan

    # Sistema
    df_sis = df_b1_f[df_b1_f["banco_nombre"] == K.NOMBRE_SISTEMA]
    def _sv(cta):
        s = df_sis[(df_sis["cuenta"]==cta)&(df_sis["periodo"]==periodo_sel)]["saldo_mes_actual
        return s.sum() if not s.empty else np.nan

    st.markdown('<div class="section-title">Indicadores del Sistema — período seleccionado</di
    c1,c2,c3,c4 = st.columns(4)
    def kcard(col, label, val, delta=""):
        with col:
```

```
        st.markdown(f"""
        <div style="background:#fff;border-radius:8px;padding:0.9rem 1.1rem;
                    box-shadow:0 2px 6px rgba(0,0,0,0.07);border-top:4px solid #F5A623">
            <div style="font-size:0.68rem;color:#888;text-transform:uppercase;
                        letter-spacing:0.05em;font-weight:600">{label}</div>
            <div style="font-size:1.3rem;font-weight:700;color:#1a1a2e;
                        font-family:'Courier New',monospace">{val}</div>
            <div style="font-size:0.8rem;margin-top:0.2rem">{delta}</div>
        </div>""", unsafe_allow_html=True)

def _sv_ref(cta):
    s = df_sis[(df_sis["cuenta"]==cta)&(df_sis["periodo"]==periodo_ref)]["saldo_mes_actual
    return s.sum() if not s.empty else np.nan
def _delta_sis(cta):
    v = _sv(cta); r = _sv_ref(cta)
    if pd.isna(v) or pd.isna(r): return ""
    d = v - r; p = d/abs(r)*100 if r!=0 else np.nan
    c = "#2E7D32" if d>0 else "#CC0000"
    a = "▲" if d>0 else "▼"
    ps = f"({cl(p,1)}%)" if pd.notna(p) else ""
    return f'<span style="color:{c};font-weight:700;font-size:0.78rem">{a} {cl(abs(d),0)}

kcard(c1, "Colocaciones Sistema",   mmm(_sv(K.CTA_COLOCACIONES)),  _delta_sis(K.CTA_COLOCA
kcard(c2, "Total Activos Sistema",  mmm(_sv(K.CTA_TOTAL_ACTIVOS)), _delta_sis(K.CTA_TOTAL_
kcard(c3, "Patrimonio Sistema",     mmm(_sv(K.CTA_PATRIMONIO)),    _delta_sis(K.CTA_PATRIM
kcard(c4, "NIM Promedio (4 bancos)",pct(float(np.nanmean([_g(nim_t,b) for b in K.BANCOS.va

st.markdown("<br>", unsafe_allow_html=True)
st.markdown('<div class="section-title">KPIs por banco</div>', unsafe_allow_html=True)
cols = st.columns(4)
for i,(cod,banco) in enumerate(K.BANCOS.items()):
    color = COLORES.get(banco,"#aaa")
    col_val  = _g(col_t, banco)
    nim_val  = _g(nim_t, banco)
    roe_val  = _g(roe_t, banco)

    # Variación según filtro global
    def _var_color(v):
        if v is None or (isinstance(v, float) and np.isnan(v)): return "#888"
        return "#2E7D32" if v > 0 else ("#CC0000" if v < 0 else "#888")
    def _var_arrow(v):
        if v is None or (isinstance(v, float) and np.isnan(v)): return "●"
        return "▲" if v > 0 else ("▼" if v < 0 else "●")

    try:
        val_ref = col_t.loc[periodo_ref, banco] if periodo_ref in col_t.index else np.nan
        delta_abs = col_val - val_ref if pd.notna(col_val) and pd.notna(val_ref) else np.n
        delta_pct = (delta_abs / abs(val_ref) * 100) if (pd.notna(delta_abs) and val_ref !
    except:
        delta_abs = np.nan
        delta_pct = np.nan

    d_color = _var_color(delta_abs)
    d_abs_s = f"{_var_arrow(delta_abs)} {cl(abs(delta_abs),0)}" if pd.notna(delta_abs) els
    d_pct_s = f"({cl(delta_pct,1)}%)" if pd.notna(delta_pct) else ""
```

```
# Mini-tabla: sin Colocaciones (ya visible arriba) y sin YoY
df_var_banco = K.tabla_variaciones(df_b1_f, df_r1_f, periodo_sel)
if not df_var_banco.empty:
    df_var_banco = df_var_banco[
        (df_var_banco["Banco"] == banco) &
        (~df_var_banco["KPI"].str.contains("Colocaciones"))
    ]
else:
    df_var_banco = pd.DataFrame()

# Construir filas de la mini-tabla (sin columna YoY)
filas_mini = ""
if not df_var_banco.empty:
    for _, vrow in df_var_banco.iterrows():
        kpi_n  = vrow["KPI"].replace(" (MMM$)","").replace(" (%)","")
        val_v  = cl(vrow["Valor"], 0)
        mom_v  = vrow["Var MoM (MMM$)"]
        momp_v = vrow["Var MoM (%)"]
        mom_s  = f'{"▲" if pd.notna(mom_v) and mom_v>0 else "▼" if pd.notna(mom_v) and
        momp_s = f'({cl(momp_v,1)}%)' if pd.notna(momp_v) else ""
        mc     = "#2E7D32" if (pd.notna(mom_v) and mom_v>0) else ("#CC0000" if pd.notna
        filas_mini += f"""<tr>
          <td style="padding:0.22rem 0.4rem;font-size:0.7rem;color:#555;border-bottom:
          <td style="padding:0.22rem 0.4rem;font-size:0.7rem;text-align:right;color:#1
          <td style="padding:0.22rem 0.4rem;font-size:0.7rem;text-align:right;color:{m
        </tr>"""

tabla_mini_html = f"""
<div style="margin-top:0.6rem;border-top:1px solid #eee;padding-top:0.4rem">
  <table style="border-collapse:collapse;width:100%">
    <thead><tr style="background:#f7f7f7">
      <th style="padding:0.2rem 0.4rem;font-size:0.62rem;color:#888;text-align:left;fo
      <th style="padding:0.2rem 0.4rem;font-size:0.62rem;color:#888;text-align:right;f
      <th style="padding:0.2rem 0.4rem;font-size:0.62rem;color:#888;text-align:right;f
    </tr></thead>
    <tbody>{filas_mini}</tbody>
  </table>
</div>""" if filas_mini else ""

with cols[i]:
    st.markdown(f"""
    <div style="background:#ffffff;border-radius:8px;padding:1rem 1.1rem;
                box-shadow:0 2px 6px rgba(0,0,0,0.10);border-left:5px solid {color};
                border:1px solid #e0e0e0">
      <div style="font-size:0.9rem;font-weight:700;color:{color};
                  text-transform:uppercase;margin-bottom:0.7rem">{banco}</div>
      <div style="margin-bottom:0.5rem">
        <div style="font-size:0.65rem;color:#666;text-transform:uppercase;font-weight:
        <div style="font-size:1.15rem;font-weight:700;color:#1a1a2e;
                    font-family:'Courier New',monospace">{mmm(col_val)}</div>
      </div>
      <div style="border-top:1px solid #eee;padding-top:0.45rem;margin-bottom:0.5rem">
        <div style="display:flex;justify-content:space-between;align-items:center">
          <span style="font-size:0.62rem;color:#888;text-transform:uppercase;font-weig
          <span style="font-size:0.82rem;font-weight:700;color:{d_color}">{d_abs_s} &n
        </div>
```

```
            </div>
            <div style="display:grid;grid-template-columns:1fr 1fr;gap:0.3rem;
                        border-top:1px solid #eee;padding-top:0.45rem">
              <div>
                <div style="font-size:0.65rem;color:#666;text-transform:uppercase;font-weigh
                <div style="font-size:1rem;font-weight:700;color:#1a1a2e">{pct(nim_val)}</di
              </div>
              <div>
                <div style="font-size:0.65rem;color:#666;text-transform:uppercase;font-weigh
                <div style="font-size:1rem;font-weight:700;color:#1a1a2e">{pct(roe_val)}</di
              </div>
            </div>
            {tabla_mini_html}
          </div>""", unsafe_allow_html=True)


# ═══════════════════════════════════════════════════════════════════════════
# TAB 2 — ACTIVOS
# ═══════════════════════════════════════════════════════════════════════════
with tab2:
    col_all    = K.colocaciones_totales(df_b1_f, incluir_sistema=True)
    col_bancos = K.colocaciones_totales(df_b1_f, incluir_sistema=False)
    act_all    = K._pivot(K._saldo(df_b1_f, K.CTA_TOTAL_ACTIVOS))

    # ── Colocaciones ──────────────────────────────────────────────────────
    st.markdown('<div class="section-title">Evolución Colocaciones — Sistema (MMM$)</div>', un
    fig_sis = go.Figure()
    if K.NOMBRE_SISTEMA in col_all.columns:
        agregar_linea(fig_sis, col_all[K.NOMBRE_SISTEMA].dropna(), K.NOMBRE_SISTEMA, dash="dot
    fig_sis.update_layout(**layout_base("Sistema Financiero (MMM$)", h=240, periodos=list(col_
    fig_sis.update_layout(margin=dict(l=70, r=80, t=45, b=40))
    st.plotly_chart(fig_sis, use_container_width=True, config=PLOT_CFG)

    st.markdown('<div class="section-title">Evolución Colocaciones — 4 Grandes Bancos (MMM$)</
    st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
    fig_b = go.Figure()
    series_col = {b: col_bancos[b].dropna() for b in _bancos_ordenados(col_bancos.columns) if 
    agregar_lineas_sin_superposicion(fig_b, series_col)
    lo_b = layout_base("Bancos (MMM$)", h=300, periodos=list(col_bancos.index), r=80)
    lo_b["showlegend"] = False
    lo_b["margin"] = dict(l=70, r=80, t=45, b=40)
    fig_b.update_layout(**lo_b)
    st.plotly_chart(fig_b, use_container_width=True, config=PLOT_CFG)

    # Mix por tipo
    st.markdown('<div class="section-title">Mix por Tipo de Colocación — período seleccionado
    df_seg = K.colocaciones_por_segmento(df_b1_f, periodo_sel)
    if not df_seg.empty:
        df_seg["texto"] = df_seg["valor"].apply(lambda x: cl(x,0))
        orden_seg = [b for b in K.ORDEN_BANCOS if b in df_seg["banco"].unique()]
        df_seg["banco"] = pd.Categorical(df_seg["banco"], categories=orden_seg, ordered=True)
        df_seg = df_seg.sort_values("banco")
        fig_seg = px.bar(df_seg, x="banco", y="valor", color="tipo", barmode="stack",
            color_discrete_map={"Comercial":"#2196F3","Vivienda":"#9C27B0","Consumo":"#FF5722"
```

```
        labels={"valor":"MMM$","banco":"","tipo":""}, text="texto")
    fig_seg.update_layout(**{**layout_base("Composición por tipo de colocación (MMM$)", h=
        "showlegend":True,
        "xaxis": dict(showgrid=False, zeroline=False, linecolor="#ddd", tickfont=dict(size
    })
    fig_seg.update_traces(textfont_size=13, textposition="outside", cliponaxis=False)
    st.plotly_chart(fig_seg, use_container_width=True, config=PLOT_CFG)

# Variación mensual — Heatmap
st.markdown('<div class="section-title">Variación Mensual Colocaciones — Heatmap (MMM$)</d
if not col_bancos.empty:
    var_mom    = col_bancos.diff(1).dropna()
    orden_heat = [b for b in K.ORDEN_BANCOS if b in var_mom.columns]
    var_pivot  = var_mom[orden_heat].T
    periodos_heat = [f"{MESES_ES.get(p.strftime('%b'), p.strftime('%b'))} {p.strftime('%y'
    z_texts = [[cl(v, 0) for v in row] for row in var_pivot.values]
    fig_heat = go.Figure(go.Heatmap(
        z=var_pivot.values.tolist(), x=periodos_heat, y=orden_heat,
        text=z_texts, texttemplate="%{text}",
        textfont=dict(size=11, color="white"),
        colorscale=[[0.0,"#CC0000"],[0.35,"#E57373"],[0.5,"#F5F5F5"],[0.65,"#81C784"],[1.0
        zmid=0, showscale=True,
        colorbar=dict(title="MMM$", tickfont=dict(size=10)),
        hovertemplate="<b>%{y}</b><br>%{x}: %{text} MMM$<extra></extra>",
    ))
    fig_heat.update_layout(paper_bgcolor="white", plot_bgcolor="white",
        font=dict(family="Open Sans", size=11), height=240,
        margin=dict(l=120, r=40, t=20, b=40),
        xaxis=dict(showgrid=False, tickfont=dict(size=10, color="#333")),
        yaxis=dict(showgrid=False, tickfont=dict(size=12, color="#1a1a2e")),
    )
    st.plotly_chart(fig_heat, use_container_width=True, config=PLOT_CFG)

# ── Activos Totales ──────────────────────────────────────────────────
st.markdown('<div class="section-title">Evolución Activos Totales — Sistema (MMM$)</div>',
fig_as = go.Figure()
if K.NOMBRE_SISTEMA in act_all.columns:
    agregar_linea(fig_as, act_all[K.NOMBRE_SISTEMA].dropna(), K.NOMBRE_SISTEMA, dash="dot"
fig_as.update_layout(**layout_base("Activos Totales — Sistema (MMM$)", h=240, periodos=lis
fig_as.update_layout(margin=dict(l=70, r=80, t=45, b=40))
st.plotly_chart(fig_as, use_container_width=True, config=PLOT_CFG)

st.markdown('<div class="section-title">Evolución Activos Totales — 4 Grandes Bancos (MMM$
st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
fig_ab = go.Figure()
series_act = {b: act_all[b].dropna() for b in _bancos_ordenados(act_all.columns) if b != K
agregar_lineas_sin_superposicion(fig_ab, series_act)
lo_ab = layout_base("Activos Totales — Bancos (MMM$)", h=300, periodos=list(act_all.index)
lo_ab["showlegend"] = False
lo_ab["margin"] = dict(l=70, r=80, t=45, b=40)
fig_ab.update_layout(**lo_ab)
st.plotly_chart(fig_ab, use_container_width=True, config=PLOT_CFG)

# Colocaciones / Activo
st.markdown('<div class="section-title">Colocaciones como % del Activo</div>', unsafe_allo
col_b1  = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_COLOCACIONES))
```

```
    act_b1  = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_TOTAL_ACTIVOS))
    pct_col = (col_b1 / act_b1 * 100).round(2)
    st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
    fig_pct = go.Figure()
    series_pct = {b: pct_col[b].dropna() for b in _bancos_ordenados(pct_col.columns) if b in p
    agregar_lineas_sin_superposicion(fig_pct, series_pct, fmt_dec=1)
    lo_pct = layout_base("Colocaciones / Activos (%)", h=300, yaxis_visible=False, periodos=li
    lo_pct["showlegend"] = False
    lo_pct["margin"] = dict(l=70, r=80, t=45, b=40)
    fig_pct.update_layout(**lo_pct)
    st.plotly_chart(fig_pct, use_container_width=True, config=PLOT_CFG)

    st.markdown("<br>", unsafe_allow_html=True)

    # ── Activos Financieros por tipo G2 ──────────────────────────────────────
    COLORES_ACT_FIN = {
        "Efectivo":              "#1A5FA8",
        "Depósitos en Bancos":   "#F5A623",
        "Cartera de Inversiones":"#2E7D32",
        "Compras con Pactos":    "#CC0000",
        "Adeudado por Bancos":   "#7B2D8B",
    }

    st.markdown('<div class="section-title">Composición Activos Financieros por Tipo — período
    df_comp_act_fin = K.composicion_activos_financieros_g2(df_b1_f, periodo_sel)
    _grafico_composicion(df_comp_act_fin, COLORES_ACT_FIN,
                         list(K.CUENTAS_ACTIVOS_FINANCIEROS_G2.keys()))

    st.markdown('<div class="section-title">Evolución Activos Financieros por Tipo (MMM$)</div
    st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
    evo_act_fin = K.evolucion_activos_financieros_g2(df_b1_f)
    cols_af = st.columns(3)
    for i, (tipo, df_evo) in enumerate(evo_act_fin.items()):
        with cols_af[i % 3]:
            fig_evo = go.Figure()
            series_evo = {b: df_evo[b].dropna()
                          for b in _bancos_ordenados(df_evo.columns) if b in df_evo.columns}
            agregar_lineas_sin_superposicion(fig_evo, series_evo)
            lo = layout_base(tipo, h=260, yaxis_visible=False,
                             periodos=list(df_evo.index), r=70)
            lo["showlegend"] = False
            fig_evo.update_layout(**lo)
            st.plotly_chart(fig_evo, use_container_width=True, config=PLOT_CFG)


# ═══════════════════════════════════════════════════════════════════════════
# TAB PASIVOS
# ═══════════════════════════════════════════════════════════════════════════
with tab_pas:

    # ── Colores por categoría ────────────────────────────────────────────────
    COLORES_DEP = {"Vistas": "#1A5FA8", "DAP": "#F5A623", "Ahorro": "#2E7D32"}
    COLORES_FONDEO = {
        "Ventas con Pacto":        "#1A5FA8",
        "Obligaciones con Bancos": "#F5A623",
        "Letras Hipotecarias":     "#2E7D32",
```

```
    "Bonos Corrientes":        "#CC0000",
    "Bonos Subordinados":      "#7B2D8B",
}
COLORES_PAS_PERM = {
    "Otros Pasivos": "#1A5FA8",
    "Provisiones":   "#F5A623",
    "Patrimonio":    "#2E7D32",
}

# ═══════════════════════════════════════════════════════════════════════
# BLOQUE 1 — Depósitos (Vista / DAP / Ahorro)
# ═══════════════════════════════════════════════════════════════════════
st.markdown('<div class="section-title">Composición Depósitos por Tipo — período seleccion
df_comp_dep = K.composicion_depositos_e2(df_b1_f, periodo_sel)
_grafico_composicion(df_comp_dep, COLORES_DEP, list(K.CUENTAS_DEPOSITOS_E2.keys()))

st.markdown('<div class="section-title">Evolución Depósitos por Tipo (MMM$)</div>', unsafe_
st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
evo_dep = K.evolucion_depositos_e2(df_b1_f)
cols_dep = st.columns(len(evo_dep))
for i, (tipo, df_evo) in enumerate(evo_dep.items()):
    with cols_dep[i]:
        fig_evo = go.Figure()
        series_evo = {b: df_evo[b].dropna() for b in _bancos_ordenados(df_evo.columns) if 
        agregar_lineas_sin_superposicion(fig_evo, series_evo)
        lo = layout_base(tipo, h=260, yaxis_visible=False, periodos=list(df_evo.index), r=
        lo["showlegend"] = False
        fig_evo.update_layout(**lo)
        st.plotly_chart(fig_evo, use_container_width=True, config=PLOT_CFG)

st.markdown("<br>", unsafe_allow_html=True)

# ═══════════════════════════════════════════════════════════════════════
# BLOQUE 2 — Fondeo de Mercados (G2)
# ═══════════════════════════════════════════════════════════════════════
st.markdown('<div class="section-title">Composición Fondeo de Mercados por Tipo — período 
df_comp_fondeo = K.composicion_fondeo_g2(df_b1_f, periodo_sel)
_grafico_composicion(df_comp_fondeo, COLORES_FONDEO, list(K.CUENTAS_FONDEO_G2.keys()))

st.markdown('<div class="section-title">Evolución Fondeo de Mercados por Tipo (MMM$)</div>
st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
evo_fondeo = K.evolucion_fondeo_g2(df_b1_f)
cols_f = st.columns(3)
for i, (tipo, df_evo) in enumerate(evo_fondeo.items()):
    with cols_f[i % 3]:
        fig_evo = go.Figure()
        series_evo = {b: df_evo[b].dropna() for b in _bancos_ordenados(df_evo.columns) if 
        agregar_lineas_sin_superposicion(fig_evo, series_evo)
        lo = layout_base(tipo, h=240, yaxis_visible=False, periodos=list(df_evo.index), r=
        lo["showlegend"] = False
        fig_evo.update_layout(**lo)
        st.plotly_chart(fig_evo, use_container_width=True, config=PLOT_CFG)

st.markdown("<br>", unsafe_allow_html=True)

# ═══════════════════════════════════════════════════════════════════════
```

```
# BLOQUE 3 — Pasivos Permanentes (Otros Pasivos / Provisiones / Patrimonio)
# ========================================================================
st.markdown('<div class="section-title">Composición Pasivos Permanentes por Tipo — período
df_comp_perm = K.composicion_pasivos_permanentes_g2(df_b1_f, periodo_sel)
_grafico_composicion(df_comp_perm, COLORES_PAS_PERM, list(K.CUENTAS_PASIVOS_PERMANENTES_G2

st.markdown('<div class="section-title">Evolución Pasivos Permanentes por Tipo (MMM$)</div
st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
evo_perm = K.evolucion_pasivos_permanentes_g2(df_b1_f)
cols_perm = st.columns(3)
for i, (tipo, df_evo) in enumerate(evo_perm.items()):
    with cols_perm[i % 3]:
        fig_evo = go.Figure()
        series_evo = {b: df_evo[b].dropna() for b in _bancos_ordenados(df_evo.columns) if
        agregar_lineas_sin_superposicion(fig_evo, series_evo)
        lo = layout_base(tipo, h=260, yaxis_visible=False, periodos=list(df_evo.index), r=
        lo["showlegend"] = False
        fig_evo.update_layout(**lo)
        st.plotly_chart(fig_evo, use_container_width=True, config=PLOT_CFG)

st.markdown("<br>", unsafe_allow_html=True)

# ========================================================================
# BLOQUE 4 — Gráficos legacy (depósitos totales + % pasivo)
# ========================================================================
dep_all    = K._pivot(K._saldo(df_b1_f, K.CTA_DEPOSITOS))
dep_bancos = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_DEPOSITOS))
pas_all    = K._pivot(K._saldo(df_b1_f, K.CTA_TOTAL_PASIVOS))

st.markdown('<div class="section-title">Evolución Depósitos — Sistema (MMM$)</div>', unsaf
fig_dep_sis = go.Figure()
if K.NOMBRE_SISTEMA in dep_all.columns:
    agregar_linea(fig_dep_sis, dep_all[K.NOMBRE_SISTEMA].dropna(),
                  K.NOMBRE_SISTEMA, dash="dot", width=2, size=4)
fig_dep_sis.update_layout(**layout_base("Depósitos — Sistema (MMM$)", h=240,
                                        periodos=list(dep_all.index), r=80))
fig_dep_sis.update_layout(margin=dict(l=70, r=80, t=45, b=40))
st.plotly_chart(fig_dep_sis, use_container_width=True, config=PLOT_CFG)

st.markdown('<div class="section-title">Evolución Depósitos — 4 Grandes Bancos (MMM$)</div
st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
fig_dep_b = go.Figure()
series_dep = {b: dep_bancos[b].dropna() for b in _bancos_ordenados(dep_bancos.columns) if
agregar_lineas_sin_superposicion(fig_dep_b, series_dep)
lo_dep = layout_base("Depósitos — Bancos (MMM$)", h=300, periodos=list(dep_bancos.index),
lo_dep["showlegend"] = False
lo_dep["margin"] = dict(l=70, r=80, t=45, b=40)
fig_dep_b.update_layout(**lo_dep)
st.plotly_chart(fig_dep_b, use_container_width=True, config=PLOT_CFG)

st.markdown('<div class="section-title">Depósitos por banco — período seleccionado (MMM$)<
if not dep_bancos.empty and periodo_sel in dep_bancos.index:
    dep_sel = dep_bancos.loc[periodo_sel].reindex(
        [b for b in K.ORDEN_BANCOS if b in dep_bancos.columns])
    fig_dep_bar = go.Figure(go.Bar(
        x=dep_sel.index, y=dep_sel.values,
```

```
            marker_color=[COLORES.get(b, "#aaa") for b in dep_sel.index],
            text=[cl(v, 0) for v in dep_sel.values],
            textposition="outside", textfont=dict(size=13, color="#1a1a2e"),
        ))
        fig_dep_bar.update_layout(**{
            **layout_base("Depósitos por banco (MMM$)", h=320),
            "showlegend": False,
            "xaxis": dict(showgrid=False, zeroline=False, linecolor="#ddd",
                          tickfont=dict(size=13, color="#1a1a2e")),
            "yaxis": dict(visible=False),
        })
        fig_dep_bar.update_traces(cliponaxis=False)
        st.plotly_chart(fig_dep_bar, use_container_width=True, config=PLOT_CFG)

    st.markdown('<div class="section-title">Depósitos como % del Pasivo Total</div>', unsafe_a
    dep_b1  = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_DEPOSITOS))
    pas_b1  = K._pivot(K._saldo(K._solo_bancos(df_b1_f), K.CTA_TOTAL_PASIVOS))
    pct_dep = (dep_b1 / pas_b1 * 100).round(2)
    st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
    fig_pct_dep = go.Figure()
    series_pct_dep = {b: pct_dep[b].dropna() for b in _bancos_ordenados(pct_dep.columns) if b :
    agregar_lineas_sin_superposicion(fig_pct_dep, series_pct_dep, fmt_dec=1)
    lo_pct_dep = layout_base("Depósitos / Pasivo Total (%)", h=300, yaxis_visible=False,
                             periodos=list(pct_dep.index), r=80)
    lo_pct_dep["showlegend"] = False
    lo_pct_dep["margin"] = dict(l=70, r=80, t=45, b=40)
    fig_pct_dep.update_layout(**lo_pct_dep)
    st.plotly_chart(fig_pct_dep, use_container_width=True, config=PLOT_CFG)


# ============================================================================
# TAB 3 — BALANCE (solo tablas)
# ============================================================================
with tab3:
    st.markdown(f'<div class="section-title">Balance Activos — Saldos y Variación {delta_lbl}
    df_act_tab = K.tabla_balance_activos(df_b1_f, periodo_sel, periodo_ref)
    _render_tabla_balance(df_act_tab, delta_lbl)

    st.markdown("<br>", unsafe_allow_html=True)
    st.markdown(f'<div class="section-title">Balance Pasivos — Saldos y Variación {delta_lbl}
    df_pas_tab = K.tabla_balance_pasivos(df_b1_f, periodo_sel, periodo_ref)
    _render_tabla_balance(df_pas_tab, delta_lbl)


# ============================================================================
# TAB 3B — RESULTADOS
# ============================================================================
with tab3b:
    st.markdown('<div class="section-title">Estado de Resultados — Saldos Mensuales y Variació

    df_res_tab = K.tabla_resultados_full(df_r1_f, periodo_sel, periodo_ref)

    if not df_res_tab.empty:
        _render_tabla_resultados(df_res_tab, delta_lbl)

    # ── Tabla ROF ──────────────────────────────────────────────────────────
```

```
    st.markdown("<br>", unsafe_allow_html=True)
    st.markdown('<div class="section-title">Detalle ROF — período seleccionado (MMM$)</div>', 

    df_rof_tab = K.tabla_rof(df_r1_f, periodo_sel, periodo_ref)
    if not df_rof_tab.empty:
        _render_tabla_resultados(df_rof_tab, delta_lbl)

    # ── Gráficos Cartera de Inversiones ──────────────────────────────────────
    st.markdown("<br>", unsafe_allow_html=True)
    st.markdown('<div class="section-title">Evolución Cartera de Inversiones por Tipo (MMM$)</

    evo_cartera = K.evolucion_cartera_inversiones(df_b1_f)
    nombres_cartera = {
        "DPV":        "Cartera Disponible para la Venta (MMM$)",
        "Negociación":"Cartera Negociación (MMM$)",
        "VCTO":       "Cartera Vencimiento (MMM$)",
    }
    st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
    for tipo in ["DPV", "Negociación", "VCTO"]:
        tabla = evo_cartera.get(tipo)
        if tabla is None or tabla.empty: continue
        fig_ci = go.Figure()
        periodos_ci = list(tabla.index)
        series_ci = {b: tabla[b].dropna() for b in _bancos_ordenados(tabla.columns) if b != K.
        agregar_lineas_sin_superposicion(fig_ci, series_ci)
        lo_ci = layout_base(nombres_cartera[tipo], h=320, periodos=periodos_ci, r=80)
        lo_ci["showlegend"] = False
        lo_ci["margin"] = dict(l=70, r=80, t=60, b=40)
        fig_ci.update_layout(**lo_ci)
        st.plotly_chart(fig_ci, use_container_width=True, config=PLOT_CFG)


# ═══════════════════════════════════════════════════════════════════════════
# TAB 4 — INGRESOS
# ═══════════════════════════════════════════════════════════════════════════
with tab4:
    # Tabla de resultados
    st.markdown('<div class="section-title">Resultados Industria — período seleccionado (MMM$)
    df_res = K.tabla_resultados(df_r1_f, periodo_sel)

    if not df_res.empty:
        _render_tabla_resultados(df_res, delta_lbl)

    st.markdown("<br>", unsafe_allow_html=True)

    # Composición ingresos
    st.markdown('<div class="section-title">Composición Ingreso Operacional (MMM$)</div>', uns
    df_ing = K.composicion_ingresos(df_r1_f, periodo_sel)
    if not df_ing.empty:
        df_ing["texto"] = df_ing["valor"].apply(lambda x: cl(x,0))
        # Orden fijo de bancos
        orden_ing = [b for b in K.ORDEN_BANCOS if b in df_ing["banco_nombre"].unique()]
        df_ing["banco_nombre"] = pd.Categorical(df_ing["banco_nombre"], categories=orden_ing, 
        df_ing = df_ing.sort_values("banco_nombre")
        fig_ing = px.bar(df_ing, x="banco_nombre", y="valor", color="componente",
            barmode="stack",
```

```
            color_discrete_map={
                "Margen Intereses":"#1A5FA8","Margen Reajustes":"#F5A623",
                "Comisiones Netas":"#888888","ROF":"#BBBBBB"},
            labels={"valor":"MMM$","banco_nombre":"","componente":""},
            text="texto")
        fig_ing.update_layout(**{**layout_base("Ingreso operacional por componente (MMM$)", h=
            "showlegend":True,
            "xaxis": dict(showgrid=False, zeroline=False, linecolor="#ddd",
                          tickfont=dict(size=13,color="#1a1a2e")),
        })
        fig_ing.update_traces(textfont_size=12, textposition="inside", insidetextanchor="middl
        st.plotly_chart(fig_ing, use_container_width=True, config=PLOT_CFG)

    # Evolución ingresos mensuales
    st.markdown('<div class="section-title">Evolución Mensual de Ingresos (MMM$)</div>', unsaf
    evo_ing = K.evolucion_ingresos(df_r1_f)
    st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
    ci1, ci2 = st.columns(2)
    for idx, (nombre, tabla) in enumerate(evo_ing.items()):
        if tabla.empty: continue
        fig_c = go.Figure()
        series_ing = {b: tabla[b].dropna() for b in _bancos_ordenados(tabla.columns) if b in t
        agregar_lineas_sin_superposicion(fig_c, series_ing)
        periodos_inc = list(tabla.index) if not tabla.empty else []
        lo_c = layout_base(f"{nombre} (MMM$)", h=300, periodos=periodos_inc, r=80)
        lo_c["showlegend"] = False
        lo_c["margin"] = dict(l=70, r=80, t=45, b=40)
        fig_c.update_layout(**lo_c)
        (ci1 if idx%2==0 else ci2).plotly_chart(fig_c, use_container_width=True, config=PLOT_C


# ==========================================================================
# TAB 5 — RENTABILIDAD
# ==========================================================================
with tab5:
    nim = K.calcular_nim(df_b1_f, df_r1_f)
    roa = K.calcular_roa(df_b1_f, df_r1_f)
    roe = K.calcular_roe(df_b1_f, df_r1_f)
    ef  = K.calcular_eficiencia(df_r1_f)

    # KPIs resumen
    st.markdown('<div class="section-title">KPIs de rentabilidad — período seleccionado</div>'
    r1c, r2c, r3c, r4c = st.columns(4)
    for col_obj, tabla, label in [(r1c,nim,"NIM"),(r2c,roa,"ROA"),(r3c,roe,"ROE"),(r4c,ef,"Efi
        with col_obj:
            st.markdown(f"**{label}**")
            for banco in list(K.BANCOS.values()):
                val = tabla.loc[periodo_sel, banco] if (
                    periodo_sel in tabla.index and banco in tabla.columns) else np.nan
                color = COLORES.get(banco,"#aaa")
                st.markdown(
                    f'<div style="display:flex;justify-content:space-between;'
                    f'padding:0.25rem 0;border-bottom:1px solid #f0f0f0">'
                    f'<span style="color:{color};font-weight:700;font-size:0.82rem">{banco}</s
                    f'<span style="font-weight:700;font-size:0.85rem">{pct(val)}</span>'
                    f'</div>', unsafe_allow_html=True)
```

```
# Nota fórmulas
st.markdown("""
<div style="background:#f8f9fa;border-left:3px solid #F5A623;padding:0.6rem 1rem;
            margin-top:0.8rem;border-radius:0 4px 4px 0;font-size:0.78rem;color:#555">
<b style="color:#1a1a2e">Fórmulas:</b>  
<b>NIM</b> = Margen Intereses × 12 / Activos  · 
<b>ROA</b> = Resultado × 12 / Activos  · 
<b>ROE</b> = Resultado × 12 / Patrimonio  · 
<b>Eficiencia</b> = Gastos Operac. / Ingresos Operac. <i>(menor = mejor)</i>
</div>
""", unsafe_allow_html=True)

st.markdown("<br>", unsafe_allow_html=True)
st.markdown('<div class="section-title">Evolución indicadores de rentabilidad</div>', unsa

nim_full = K.calcular_nim(df_b1, df_r1)
roa_full = K.calcular_roa(df_b1, df_r1)
roe_full = K.calcular_roe(df_b1, df_r1)
ef_full  = K.calcular_eficiencia(df_r1)

def _fig_interanual(tabla_full, titulo):
    p_2y = periodo_sel - pd.DateOffset(years=2)
    p_1y = periodo_sel - pd.DateOffset(years=1)
    def _lbl(p):
        return f"{MESES_ES.get(p.strftime('%b'), p.strftime('%b'))} {p.strftime('%y')}"
    puntos = [p_2y, p_1y, periodo_sel]
    etiquetas = [_lbl(p) for p in puntos]
    fig = go.Figure()
    for banco in [b for b in K.ORDEN_BANCOS if b in tabla_full.columns]:
        vals  = [tabla_full.loc[p, banco] if p in tabla_full.index else np.nan for p in pu
        texts = [cl(v, 2) + "%" if pd.notna(v) else "" for v in vals]
        color = COLORES.get(banco, "#aaa")
        tpos  = ["middle left"] + ["top center"] * (len(vals) - 2) + ["middle right"]
        fig.add_trace(go.Scatter(
            x=etiquetas, y=vals, name=banco,
            mode="lines+markers+text",
            line=dict(color=color, width=2.5),
            marker=dict(size=8, color=color),
            text=texts, textposition=tpos,
            textfont=dict(size=11, color=color),
            showlegend=False,
            hovertemplate=f"<b>{banco}</b><br>%{{x}}: %{{y:.2f}}%<extra></extra>",
        ))
    fig.update_layout(
        paper_bgcolor="white", plot_bgcolor="white",
        font=dict(family="Open Sans", size=11, color="#1a1a2e"),
        height=300, showlegend=False,
        margin=dict(l=80, r=90, t=45, b=40),
        title=dict(text=f"<b>{titulo}</b>", font=dict(size=12, color="#1a1a2e"), x=0),
        xaxis=dict(showgrid=False, zeroline=False, linecolor="#ddd", tickfont=dict(size=12
        yaxis=dict(visible=False, showgrid=False, zeroline=False),
    )
    return fig

st.markdown(_leyenda_bancos(), unsafe_allow_html=True)
```

```
    metricas_r = [
        ("NIM — Margen Neto de Intereses (%)", nim, nim_full, "NIM interanual (%)"),
        ("ROA — Retorno sobre Activos (%)",    roa, roa_full, "ROA interanual (%)"),
        ("ROE — Retorno sobre Patrimonio (%)", roe, roe_full, "ROE interanual (%)"),
        ("Índice de Eficiencia (%) — menor = mejor", ef, ef_full, "Eficiencia interanual (%)")
    ]
    for titulo, tabla, tabla_full, titulo_inter in metricas_r:
        col_izq, col_der = st.columns(2)
        with col_izq:
            fig_r = go.Figure()
            series_rent = {b: tabla[b].dropna() for b in _bancos_ordenados(tabla.columns) if b
            agregar_lineas_sin_superposicion(fig_r, series_rent, fmt_dec=2)
            periodos_r = list(tabla.index) if not tabla.empty else []
            lo_r = layout_base(titulo, h=300, yaxis_visible=False, periodos=periodos_r, r=110)
            lo_r["showlegend"] = False
            lo_r["margin"] = dict(l=80, r=110, t=45, b=40)
            fig_r.update_layout(**lo_r)
            st.plotly_chart(fig_r, use_container_width=True, config=PLOT_CFG)
        with col_der:
            st.plotly_chart(_fig_interanual(tabla_full, titulo_inter), use_container_width=Tru


# ═══════════════════════════════════════════════════════════════════════════
# TAB 7 — PREDICCIÓN NIM (LSTM)
# ═══════════════════════════════════════════════════════════════════════════
with tab7:
    st.markdown('''
    <div class="section-title">Predicción NIM — Modelo LSTM Multivariado</div>
    ''', unsafe_allow_html=True)

    # Verificar si los modelos ya fueron entrenados
    if not NP.modelos_disponibles():
        st.warning(
            "⚠️ Los modelos aún no han sido entrenados. "
            "Ejecuta en el Anaconda Prompt:\n\n"
            "```\npython nim_predictor.py\n```"
        )
        st.info(
            "Esto tarda ~1 minuto y solo necesitas hacerlo una vez (o cuando lleguen datos nue
            "Los resultados se guardan en `data/nim_models/` y el dashboard los carga instantá
        )
    else:
        resultados_nim = NP.cargar_resultados()

        if not resultados_nim:
            st.error("No se pudieron cargar los resultados del modelo.")
        else:
            # ── Selector de banco ──────────────────────────────────────
            bancos_disp = list(resultados_nim.keys())
            banco_sel   = st.selectbox("Seleccionar banco", bancos_disp,
                                       key="nim_banco_sel")
            r = resultados_nim[banco_sel]

            # ── Métricas resumen ───────────────────────────────────────
            st.markdown('<div class="section-title">Métricas del modelo</div>',
                        unsafe_allow_html=True)
```

```
mc1, mc2, mc3, mc4 = st.columns(4)
mc1.metric("MAE",  f"{r['mae']:.3f}%",
            help="Error Absoluto Medio: desviación promedio de la predicción")
mc2.metric("RMSE", f"{r['rmse']:.3f}%",
            help="Raíz del Error Cuadrático Medio: penaliza errores grandes")
mc3.metric("MAPE", f"{r['mape']:.1f}%",
            help="Error porcentual medio (puede inflarse con valores pequeños)")
mc4.metric(f"Predicción {r['prox_periodo']}",
            f"{r['prox_nim']:.2f}%",
            help="NIM estimado para el próximo mes")

st.markdown("<br>", unsafe_allow_html=True)

# ── Gráfico histórico + predicciones ──────────────────────
st.markdown('<div class="section-title">Serie histórica y predicción</div>',
            unsafe_allow_html=True)

periodos_hist_all = pd.to_datetime(r["periodos_hist"])
nim_hist_all      = np.array(r["nim_hist"])
# Mostrar histórico solo desde 2025
mask_2025         = periodos_hist_all >= pd.Timestamp("2025-01-01")
periodos_hist     = periodos_hist_all[mask_2025]
nim_hist          = nim_hist_all[mask_2025]
periodos_test = pd.to_datetime(r["periodos_test"])
y_real        = r["y_real"]
y_pred        = r["y_pred"]
prox_periodo  = pd.Timestamp(r["prox_periodo"])
prox_nim      = r["prox_nim"]

color_banco = {
    "BancoEstado":    "#F5A623",
    "Banco de Chile": "#1A5FA8",
    "Santander":      "#CC0000",
    "BCI":            "#2E7D32",
}.get(banco_sel, "#888")

fig_nim = go.Figure()

# Etiquetas primer y último punto
def _etiquetas(vals):
    txts = [""] * len(vals)
    if len(vals) > 0:
        txts[0]  = f"{vals[0]:.2f}%"
        txts[-1] = f"{vals[-1]:.2f}%"
    return txts

# Serie histórica completa (fondo)
fig_nim.add_trace(go.Scatter(
    x=periodos_hist, y=nim_hist,
    name="Histórico",
    mode="lines+text",
    line=dict(color=color_banco, width=1.5, dash="dot"),
    opacity=0.5,
    text=_etiquetas(nim_hist),
    textposition="top center",
    textfont=dict(size=11, color=color_banco),
```

```
))

# Valores reales en el período de test
fig_nim.add_trace(go.Scatter(
    x=periodos_test, y=y_real,
    name="Real (test)",
    mode="lines+markers+text",
    line=dict(color="#2E7D32", width=2.5),
    marker=dict(size=6),
    text=_etiquetas(y_real),
    textposition="top center",
    textfont=dict(size=11, color="#2E7D32"),
    hovertemplate="%{x|%b %Y}: %{y:.2f}%<extra>Real</extra>",
))

# Predicciones del modelo en el test
fig_nim.add_trace(go.Scatter(
    x=periodos_test, y=y_pred,
    name="Predicho (LSTM)",
    mode="lines+markers+text",
    line=dict(color="#E53935", width=2.5, dash="dash"),
    marker=dict(size=6, symbol="square"),
    text=_etiquetas(y_pred),
    textposition="bottom center",
    textfont=dict(size=11, color="#E53935"),
    hovertemplate="%{x|%b %Y}: %{y:.2f}%<extra>Predicho</extra>",
))

# Predicción del próximo mes
fig_nim.add_trace(go.Scatter(
    x=[prox_periodo], y=[prox_nim],
    name=f"Próximo mes ({r['prox_periodo']})",
    mode="markers+text",
    marker=dict(size=14, color="purple", symbol="diamond"),
    text=[f"{prox_nim:.2f}%"],
    textposition="top center",
    textfont=dict(size=12, color="purple"),
    hovertemplate=f"{r['prox_periodo']}: {prox_nim:.2f}%<extra>Predicción</extra>"
))

# Línea vertical separando train/test
if len(periodos_test) > 0:
    fig_nim.add_vline(
        x=periodos_test[0].timestamp() * 1000,
        line_dash="dot", line_color="gray", line_width=1,
        annotation_text="← train | test →",
        annotation_position="top",
        annotation_font_size=11,
    )

fig_nim.update_layout(
    **layout_base(f"NIM Mensual Anualizado — {banco_sel}", h=420,
                  yaxis_visible=True, yaxis_suffix="%"),
)
fig_nim.update_yaxes(showgrid=False, visible=False)
st.plotly_chart(fig_nim, use_container_width=True, config=PLOT_CFG)
```

```
            # ── Comparativa todos los bancos ──────────────────────────────
            st.markdown('<div class="section-title">Predicción próximo mes — todos los bancos<
                        unsafe_allow_html=True)

            df_prox = pd.DataFrame([
                {"Banco": nombre, "NIM actual (último mes)": vals["nim_hist"][-1],
                 f"NIM predicho ({vals['prox_periodo']})": vals["prox_nim"],
                 "MAE modelo": vals["mae"]}
                for nombre, vals in resultados_nim.items()
            ])

            # Gráfico de barras comparativo
            fig_comp = go.Figure()
            colores_bancos = {"BancoEstado": "#F5A623", "Banco de Chile": "#1A5FA8",
                              "Santander": "#CC0000", "BCI": "#2E7D32"}

            fig_comp.add_trace(go.Bar(
                x=df_prox["Banco"],
                y=df_prox["NIM actual (último mes)"],
                name="Último mes real",
                marker_color=[colores_bancos.get(b, "#888") for b in df_prox["Banco"]],
                opacity=0.5,
                text=[f"{v:.2f}%" for v in df_prox["NIM actual (último mes)"]],
                textposition="outside",
            ))
            fig_comp.add_trace(go.Bar(
                x=df_prox["Banco"],
                y=df_prox[f"NIM predicho ({list(resultados_nim.values())[0]['prox_periodo']})"
                name="Predicción próximo mes",
                marker_color=[colores_bancos.get(b, "#888") for b in df_prox["Banco"]],
                text=[f"{v:.2f}%" for v in df_prox[f"NIM predicho ({list(resultados_nim.values
                textposition="outside",
            ))

            fig_comp.update_layout(
                **layout_base("Comparativa NIM: último mes vs predicción", h=360,
                              yaxis_visible=True, yaxis_suffix="%"),
                barmode="group",
            )
            fig_comp.update_yaxes(showgrid=True, gridcolor="#f0f0f0", ticksuffix="%")
            fig_comp.update_xaxes(showgrid=False, zeroline=False,
                                  tickfont=dict(size=13, color="#1a1a2e"))
            st.plotly_chart(fig_comp, use_container_width=True, config=PLOT_CFG)

            # ── Nota metodológica ─────────────────────────────────────────
            with st.expander("ℹ️ Metodología del modelo"):
                st.markdown("""
**Modelo:** LSTM Multivariado (Long Short-Term Memory)

**Variables de entrada:**
- NIM propio del banco (desacumulado, anualizado)
- TPM — Tasa de Política Monetaria (BCCh via mindicador.cl)
- IPC — Variación mensual del IPC (INE via mindicador.cl)
- Spread del sistema financiero (CMF, código 999)
```

```
**Arquitectura:** 2 capas LSTM (64 + 32 unidades) + Dropout(0.2) + Dense(1)

**Ventana temporal:** 6 meses de historia para predecir el mes siguiente

**Entrenamiento:** EarlyStopping sobre val_loss, patience=20

**Datos:** CMF Chile · {len(r["periodos_hist"])} períodos disponibles
                """)


# FOOTER
st.markdown("---")
st.markdown("<small style='color:#94a9be'>Fuente: CMF Chile · Datos públicos · "
            "Uso académico — Diplomado Ciencia de Datos en Finanzas · FEN UCHILE</small>",
            unsafe_allow_html=True)
```